WFLOW-99: Best Practice Summary

> **Series:** WFLOW | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/26/2026

Overview

This notebook consolidates every actionable best practice from the WFLOW series (notebooks 01-09) into definitive tables organized by category. Each best practice includes the exact setting or value to use, a priority rating, and the source notebook.

Table of Contents

---

1. Workflow Design & Architecture

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use Workflows for all new automation	Workflows (not legacy alerting profiles)	Critical	WFLOW-01
2	Stay within execution time limit	Max 15 minutes per workflow execution	Critical	WFLOW-01
3	Stay within task limit	Max 50 tasks per workflow	Critical	WFLOW-01
4	Set individual task timeouts	5 minutes max per task	Recommended	WFLOW-01
5	Respect concurrent execution cap	Max 100 concurrent executions per environment	Critical	WFLOW-01
6	Start with simple notifications	Basic notifications first, then layer in complexity	Recommended	WFLOW-09
7	Run notification tasks in parallel	Default parallel execution for independent tasks (Slack + Teams + Email simultaneously)	Recommended	WFLOW-03
8	Use sequential execution with `dependsOn`	Chain tasks that require previous task output	Recommended	WFLOW-04

2. Triggers

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use Detected Problem trigger for incident alerts	`type: davis-problem`	Critical	WFLOW-02
2	Use Detected Event trigger for proactive metric thresholds	`type: davis-event` with DQL query and `evaluationFrequency: "5m"`	Critical	WFLOW-02
3	Use Schedule trigger for reports and health checks	`type: schedule` with cron expression and explicit `timezone`	Critical	WFLOW-02

#	Best Practice	Recommended Setting/Value	Priority	Source
4	Use On-Demand trigger for testing	`type: on-demand` before deploying any production workflow	Critical	WFLOW-02
5	Scope Detected Problem triggers with entity tags	`entityTagsMatch: all` with `entityTags: [{key: env, value: prod}]`	Critical	WFLOW-02
6	Filter by problem categories	Set `categories: [AVAILABILITY, PERFORMANCE]` to reduce noise	Recommended	WFLOW-02
7	Handle all problem lifecycle events	Trigger on OPEN, UPDATED, and CLOSED statuses in a single workflow	Recommended	WFLOW-05
8	Offset scheduled workflows from minute :00	Use `:05`, `:15`, `:30` instead of `:00` to spread load	Recommended	WFLOW-02
9	Always set timezone on schedule triggers	`timezone: "America/New_York"` (explicit, never rely on default)	Recommended	WFLOW-02
10	Use Event trigger for business process automation	`type: event` with `filterQuery` for bizevent-driven workflows	Optional	WFLOW-02

3. Connections & Credentials

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use descriptive connection names	Pattern: `<service>-<environment>-<purpose>` (e.g., `slack-prod-oncall`)	Critical	WFLOW-03
2	Separate connections per environment	Distinct connections for prod vs staging vs dev	Critical	WFLOW-03
3	Apply least-privilege scopes	Minimum OAuth scopes required (e.g., Slack: `chat:write`, `chat:write.public` only)	Critical	WFLOW-03

#	Best Practice	Recommended Setting/Value	Priority	Source
4	Rotate tokens on a schedule	Monthly rotation of API tokens and passwords	Recommended	WFLOW-09
5	Use OAuth 2.0 over Basic Auth for ServiceNow	`OAuth 2.0` for production; Basic Auth only for dev/testing	Recommended	WFLOW-05
6	Create separate PagerDuty connections per service routing	`pagerduty-prod-critical`, `pagerduty-prod-standard`, `pagerduty-platform`	Recommended	WFLOW-05
7	Prefer Slack OAuth App over Incoming Webhook	OAuth App provides channels, DMs, reactions, threads	Recommended	WFLOW-03

4. Notification Channels

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Multi-channel for Critical production problems	PagerDuty + Slack #urgent + Email simultaneously	Critical	WFLOW-03, WFLOW-04
2	Slack for High production problems	Slack #alerts + Email	Recommended	WFLOW-04
3	Slack-only for Medium problems	Slack #alerts (business hours only for Low)	Recommended	WFLOW-04
4	Use Power Automate connector for Teams	Microsoft is deprecating Office 365 Connectors; use newer Workflows connector	Recommended	WFLOW-03
5	Use built-in email for simple notifications	No SMTP setup required; use custom SMTP or SendGrid for high volume	Optional	WFLOW-03
6	Include severity in email subject line	`[{{ event()['severity'] }}] {{ event()['title'] }}`	Recommended	WFLOW-03

5. Message Formatting & Templates

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Structure messages: Severity + Title + Key metrics + Link	Line 1: severity indicator + title; Line 2: impact; Line 3: affected entities; Line 4: action link	Critical	WFLOW-06
2	Use severity-coded visual indicators	Slack: `:red_circle:` CRITICAL, `:large_orange_circle:` HIGH, `:large_yellow_circle:` MEDIUM, `:large_blue_circle:` LOW	Critical	WFLOW-06
3	Use Slack Block Kit for rich messages	`blocks:` with `header`, `section`, `actions`, `context` types	Recommended	WFLOW-06
4	Use Teams Adaptive Cards v1.4	`type: AdaptiveCard`, `version: "1.4"` with `FactSet`, `TextBlock`, `Action.OpenUrl`	Recommended	WFLOW-06
5	Always include a link to the Dynatrace problem	`<{{ event()['problem_url'] }}|View in Dynatrace>` (Slack) or `Action.OpenUrl` (Teams)	Critical	WFLOW-03
6	Use dictionary mapping for severity-to-emoji	`{{ {"CRITICAL": ":red_circle:", ...}.get(event()["severity"], ":white_circle:") }}`	Recommended	WFLOW-06
7	Limit affected entities shown to 3-5	`event()["affected_entity_ids"][:3]` with overflow indicator	Recommended	WFLOW-06

#	Best Practice	Recommended Setting/Value	Priority	Source
8	Use `.get()` with defaults for optional fields	`event().get("root_cause_entity_id", "Pending analysis")`	Critical	WFLOW-06
9	Truncate long strings	`{{ event()["title"] | truncate(50) }}`	Recommended	WFLOW-06
10	Enrich notifications with DQL data	Add recent error logs via `queryExecutionClient.queryExecute()` in a JavaScript task before the notification task	Optional	WFLOW-06
11	Test templates with mock data first	On-Demand trigger + JavaScript mock event task before production deployment	Critical	WFLOW-06
12	Preview Slack templates in Block Kit Builder	https://app.slack.com/block-kit-builder before deploying	Recommended	WFLOW-06
13	Create a resolved-problem template	`:white_check_mark: Problem Resolved` with duration and resolution time	Recommended	WFLOW-06

6. Routing & Escalation

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Route by severity	CRITICAL: PagerDuty + Slack + Email; HIGH: Slack + Email; MEDIUM: Slack; LOW: Slack business hours only	Critical	WFLOW-04

#	Best Practice	Recommended Setting/Value	Priority	Source
2	Route by team ownership via entity tags	Condition: `"team:checkout" in event().get("tags", [])` maps to `#checkout-alerts`	Critical	WFLOW-04
3	Route by management zone	Condition: `"Production" in event()["management_zones"]`	Critical	WFLOW-04
4	Implement time-based routing	Business hours (Mon-Fri 9-17): Slack channel; Off-hours Critical: PagerDuty; Off-hours non-critical: queue for morning	Critical	WFLOW-04
5	Auto-escalate unacknowledged alerts	Wait 15 min, check if still OPEN, escalate to PagerDuty if unacknowledged	Recommended	WFLOW-04
6	Multi-tier escalation	0 min: Slack; 15 min: Email team lead; 30 min: PagerDuty on-call; 60 min: PagerDuty manager	Recommended	WFLOW-04
7	Use AND logic for multi-condition tasks	`conditions: [is_critical, is_production]` requires both true	Recommended	WFLOW-04
8	Use JavaScript for dynamic channel selection	Parse `team:` tag to construct `#${team}-alerts` dynamically	Optional	WFLOW-04
9	Separate environment routing	Production: immediate notification; Staging: Slack only; Dev: daily digest	Recommended	WFLOW-04

7. Incident Management Integration

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Deduplicate PagerDuty incidents	`dedupKey: "dynatrace-{{ event()['display_id'] }}"`	Critical	WFLOW-05
2	Deduplicate ServiceNow incidents	`correlation_id: "DT-{{ event()['display_id'] }}"`	Critical	WFLOW-05

#	Best Practice	Recommended Setting/Value	Priority	Source
3	Map Dynatrace severity to PagerDuty severity	CRITICAL: `critical`; HIGH: `error`; MEDIUM: `warning`; LOW: `info`	Critical	WFLOW-05
4	Map Dynatrace severity to ServiceNow impact/urgency	CRITICAL: impact=1, urgency=1; HIGH: impact=2, urgency=2; MEDIUM: impact=2, urgency=2; LOW: impact=3, urgency=3	Critical	WFLOW-05
5	Auto-resolve incidents when problem closes	Trigger on `event()["status"] == "CLOSED"`, resolve PagerDuty via `dedupKey` or set ServiceNow `state: 6`	Critical	WFLOW-05
6	Include Dynatrace problem URL in incident details	`customDetails.problem_url` (PagerDuty) or `description` body (ServiceNow)	Critical	WFLOW-05
7	Store incident ID for subsequent updates	Capture `sys_id` from create response in a JavaScript task for update/resolve operations	Recommended	WFLOW-05
8	Use PagerDuty Events API v2	Integration type: Events API v2 (not v1)	Critical	WFLOW-05
9	Set `assignment_group` and `caller_id` in ServiceNow	`assignment_group: "Platform Engineering"`, `caller_id: "dynatrace.integration"`	Recommended	WFLOW-05
10	Implement bi-directional sync	Scheduled workflow queries ServiceNow for open DT-correlated incidents and updates Dynatrace problem comments	Optional	WFLOW-05

8. Auto-Remediation

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Rate-limit remediation attempts	Max 3 remediations per hour per entity	Critical	WFLOW-07

#	Best Practice	Recommended Setting/Value	Priority	Source
2	Enforce time-window guardrails	Remediation only during safe hours: weekdays 06:00-22:00	Critical	WFLOW-07
3	Implement cooldown periods	30 minutes minimum between remediation actions on the same entity	Critical	WFLOW-07
4	Capture pre-remediation state for rollback	Record current state before executing any change	Critical	WFLOW-07
5	Require human approval for production remediation	Slack approval buttons with 30-minute timeout; auto-approve for non-production only	Critical	WFLOW-07
6	Auto-remediate only well-defined scenarios	Pod crash loops: yes (restart); Disk space 90%: yes (cleanup); Database deadlocks: no (investigate); Security incidents: no (human judgment)	Critical	WFLOW-07
7	Follow remediation maturity model	Level 0: manual; Level 1: notification + runbook link; Level 2: semi-automated (approval); Level 3: fully automated with guardrails	Recommended	WFLOW-07
8	Map problem types to runbooks	JavaScript `runbookMap` object mapping problem title patterns to runbook IDs	Recommended	WFLOW-07
9	Include runbook links in notifications	Jinja conditional: `{% if "CPU" in event()["title"] %}` links to CPU runbook	Recommended	WFLOW-07
10	Validate after remediation	Query metrics/logs post-action to confirm resolution	Recommended	WFLOW-07
11	Start non-prod, promote to prod	Test all remediation workflows in staging before enabling in production	Critical	WFLOW-07

9. JavaScript & HTTP Actions

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Always use `export default async function`	Required function signature for every JavaScript action	Critical	WFLOW-08
2	Wrap all external calls in try-catch	Return `{success: false, error: error.message}` on failure instead of throwing	Critical	WFLOW-08
3	Implement retry with exponential backoff	Max 3 retries, delay = `1000 * attempt` ms, retry only on 5xx errors	Recommended	WFLOW-08
4	Set request timeouts	`AbortController` with 10-second timeout on external HTTP calls	Critical	WFLOW-08
5	Set DQL query timeouts	`requestTimeoutMilliseconds: 30000` on all `queryExecute()` calls	Critical	WFLOW-08
6	Limit DQL query scope	`from: now() - 1h` , select only needed `fields` , `limit 100`	Critical	WFLOW-08
7	Use `Promise.all()` for parallel entity lookups	Execute all independent API calls concurrently	Recommended	WFLOW-08
8	Always use HTTPS for external calls	Never use `http://` in HTTP request URLs	Critical	WFLOW-09
9	Access secrets via `env` object only	`env.API_TOKEN` (never hardcode, never log secrets via `console.log`)	Critical	WFLOW-08, WFLOW-09
10	Add problem comments after remediation	Use `problemsClient.createComment()` to record automated actions	Recommended	WFLOW-08
11	Validate secret existence before use	`if (!apiToken) throw new Error('Missing required secret: ...')`	Recommended	WFLOW-09

10. Security

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Never hardcode secrets in workflows	Use `{{ env.SECRET_NAME }}` in YAML or `env.SECRET_NAME` in JavaScript	Critical	WFLOW-09
2	Sanitize all dynamic inputs	Remove `"'\\` characters, limit string length to 200 chars before using in queries	Critical	WFLOW-09
3	Never log secrets	Do not use `console.log(env.API_TOKEN)` or return secrets in task output	Critical	WFLOW-09
4	Use secret naming convention	Pattern: `<SERVICE>_API_TOKEN`, `<SERVICE>_WEBHOOK_URL`, `<SERVICE>_<ENV>_CRED`	Recommended	WFLOW-09
5	Rotate secrets monthly	Create new secret, update workflow, test, remove old secret	Recommended	WFLOW-09
6	Fail secure on errors	Default to safe state (no action) when errors occur, never expose error details externally	Critical	WFLOW-09

11. Access Control

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Grant minimum permissions per role	Viewers: `automation:workflows:read`; App teams: `read` + `write` + `run` (own workflows); SRE: `read` + `write` + `run` (all); Admins: `admin`	Critical	WFLOW-09
2	Restrict workflow write to owners	IAM policy: `ALLOW automation:workflows:write WHERE workflow.owner == "${user.email}"`	Recommended	WFLOW-09

#	Best Practice	Recommended Setting/Value	Priority	Source
3	Separate connection write access	`automation:connections:write` only for workflow admins and SRE	Critical	WFLOW-09
4	Tag workflows with ownership metadata	`metadata.owner: sre-team@company.com`, `metadata.team: platform`, `metadata.classification: production`	Recommended	WFLOW-09

12. Observability & Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Alert on workflow success rate	Threshold: < 95% success rate over 7 days	Critical	WFLOW-09
2	Alert on average execution duration	Threshold: > 5 minutes average	Recommended	WFLOW-09
3	Alert on failure spike	Threshold: > 5 failures per hour	Critical	WFLOW-09
4	Build a workflow health dashboard	Queries: overall health, per-workflow success rates, execution trend, recent failures, task-level performance	Critical	WFLOW-09
5	Create a workflow-monitoring workflow	Scheduled every 15-60 min; query for workflows with 3+ failures/hour; alert to `#workflow-alerts`	Recommended	WFLOW-09
6	Monitor notification task success rates	Query `automation.task.execution` filtered by `slack`, `msteams`, `email`, `pagerduty` task types	Recommended	WFLOW-03

#	Best Practice	Recommended Setting/Value	Priority	Source
7	Track remediation success rates	Query `automation.workflow.execution` filtered by `remediation` or `auto-heal` workflow names	Recommended	WFLOW-07
8	Monitor skipped tasks	Query `task.status == "SKIPPED"` to validate routing conditions are working as designed	Optional	WFLOW-04

13. Operations & Change Management

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Name workflows consistently	Pattern: `<team>-<function>-<environment>` (e.g., `sre-problem-notifications-prod`)	Critical	WFLOW-09
2	Export workflows as JSON for version control	`GET /platform/automation/v1/workflows/<id>` saved to Git	Recommended	WFLOW-09
3	Test changes on cloned workflow first	Clone, modify, test with On-Demand trigger, review results, promote	Critical	WFLOW-09
4	Rollback procedure	Disable failing workflow (toggle off), import previous JSON, enable restored version, verify	Critical	WFLOW-09
5	Daily: check execution dashboard	Review success rates and investigate failures	Recommended	WFLOW-09
6	Weekly: verify external connections	Test all Slack, Teams, PagerDuty, ServiceNow connections	Recommended	WFLOW-09

#	Best Practice	Recommended Setting/Value	Priority	Source
7	Monthly: rotate secrets	Update API tokens, passwords, webhook URLs	Recommended	WFLOW-09
8	Query execution history with DQL	`fetch events, from: now() - 24h | filter event.type == "automation.workflow.execution"`	Recommended	WFLOW-01

Summary

This notebook contains **91 best practices** across 13 categories extracted from the complete WFLOW series.

Priority distribution:

Priority	Count	Meaning
Critical	47	Must implement for production readiness
Recommended	38	Strongly advised for operational maturity
Optional	6	Beneficial for advanced use cases

Implementation order:

1. Connections and credentials (WFLOW-03)
2. Basic problem notification workflow with severity routing (WFLOW-03, WFLOW-04)
3. Incident management integration with deduplication (WFLOW-05)
4. Custom templates with severity indicators and links (WFLOW-06)
5. Workflow health monitoring dashboard (WFLOW-09)
6. Auto-remediation with guardrails (WFLOW-07)

References

- Dynatrace Workflows Documentation
- Workflow Actions Reference

- Workflow Triggers
- Jinja Expressions
- Dynatrace SDK
